

CSA0613 DESIGN ANALYSIS OF ALGORITHMS

CO1_ASSESSMENT TOOL 1_APPLICATION_BASED QUESTIONS

Q.25) Application: Fraud Detection System A fraud detection algorithm follows the recurrence $T(n)=2T(n/2)+n^2\log n$. Apply the Master Theorem to determine the time complexity. Compare $f(n)$ with $n^{\log_b a}$ and justify the correct case. Analyze performance for large transaction datasets. Interpret efficiency clearly. For assessment

Aim:

To analyze the recurrence relation $T(n) = 2T(n/2) + n^2\log n$ using the Master Theorem and determine its time complexity for a fraud detection system.

Algorithm

Step 1: Start.

Step 2: Consider the recurrence relation:

$$T(n) = 2T(n/2) + n^2\log n$$

Step 3: Set $a = 2$, $b = 2$ and $f(n) = n^2\log(n)$.

Step 4: Compute $n^{\log_b a} = n$.

Step 5: Compare $f(n)$ with $n^{\log_b a}$.

Step 6: Since $n^2\log(n)$ grows faster than n , apply Master Theorem Case 3.

Step 7: Therefore, Time Complexity = $\Theta(n^2\log(n))$.

Step 8: Stop.

Python Program

```
1 import math
2
3 n = int(input("Enter the number of transactions: "))
4
5 value = n * n * math.log2(n)
6
7 print("Estimated Work (n²logn):", round(value, 2))
8 print("Time Complexity =  $\Theta(n^2 \log n)$ ")
```

Output

```
Output
Enter the number of transactions: 16
Estimated Work (n²logn): 1024.0
Time Complexity =  $\Theta(n^2 \log n)$ 

=== Code Execution Successful ===
```

Time Complexity

$\Theta(n^2 \log n)$

Space Complexity

$\Theta(\log n)$

Result

The recurrence $T(n) = T(n/2) + 1$ satisfies Master Theorem Case 2. Therefore, the time complexity is $\Theta(\log n)$, making it efficient for real-time fraud detection.

Q34. Application: Online Banking Fraud Detection A fraud detection system analyzes transactions recursively: $T(n) = T(n/2) + 1$. Apply the Master Theorem and determine the time complexity. Identify the appropriate case and justify. Explain how this efficiency impacts real-time fraud detection performance

Aim

To analyze the recurrence relation $T(n)=T(n/2)+1$ using the Master Theorem and determine its time complexity for online banking fraud detection.

Algorithm

Step 1: Start.

Step 2: Consider the recurrence relation:

$$T(n) = T(n/2) + 1$$

Step 3: Set $a = 1$, $b = 2$ and $f(n) = 1$.

Step 4: Compute $n^{(\log_b a)} = 1$.

Step 5: Compare $f(n)$ with $n^{(\log_b a)}$.

Step 6: Apply Master Theorem Case 2.

Step 7: Therefore, Time Complexity = $\Theta(\log n)$.

Step 8: Stop.

Python program

```
1 import math
2
3 n = int(input("Enter the number of transactions: "))
4
5 levels = math.log2(n)
6
7 print("Recursive Levels:", int(levels))
8 print("Time Complexity =  $\Theta(\log n)$ ")
```

Output

```
Output

Enter the number of transactions: 16
Recursive Levels: 4
Time Complexity =  $\Theta(\log n)$ 

=== Code Execution Successful ===
```

Time Complexity

$\Theta(\log n)$

Space Complexity

$\Theta(\log n)$

Result

The recurrence $T(n) = T(n/2) + 1$ satisfies Master Theorem Case 2. Therefore, the time complexity is $\Theta(\log n)$, making it efficient for real-time fraud detection.